

Software Architecture and Design Specification

Project: Collaborative Study Material & Quiz Platform

Version: 1.0

Authors: Dharani S, Prajwal M, Karan Varshney, Harshit Chandak

Date: 13-09-2026

Status: Final Review

Revision History

1.0 | 13-09-2026 | Team 10 | Project-specific SAD completed

Approvals

Course Coordinator | - | -

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Collaborative Study Material & Quiz Platform.

1.2 Scope

Covers student study-material sharing, quiz creation and attempt, automatic evaluation, performance viewing, and administrator moderation.

1.3 Audience

Developers, testers, project guide/instructor, evaluators, and maintainers.

1.4 Definitions

SAD, SRS, UI, API, REST, RBAC, MCQ, MERN, MongoDB.

2. Document Overview

2.1 How to use this document

Describes the architecture, components, UML diagrams, APIs, security architecture, and design decisions of the project.

2.2 Related Documents

Software Requirements Specification (SRS) and team work-division document.

3. Architecture

3.1 Goals & Constraints

Goals: modular, secure, maintainable, reliable. Constraints: MERN stack, MongoDB, PDF-only study materials, Student/Administrator roles, and a single local/development deployment.

3.2 Stakeholders & Concerns

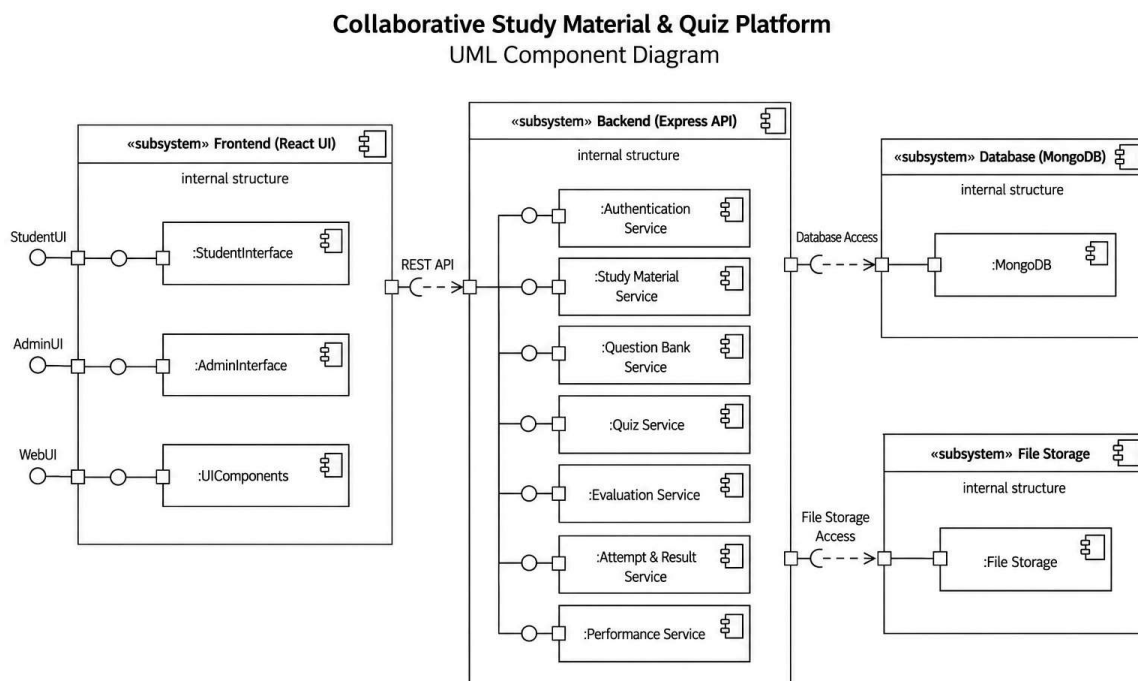
Students: usability, access to materials, reliable quizzes

Administrators: moderation and control

Developers: modularity, maintainability, integration

Instructors/Evaluators: correctness, traceability, testability.

3.3 Component (UML) Diagram



3.4 Component Descriptions

- React UI: student and administrator web interface.
- Express API: REST endpoints, validation, business logic.
- Auth/RBAC: authentication and administrator authorization.
- Study Material, Question Bank, Quiz/Evaluation modules: core application features.
- MongoDB/File Storage: persistent data and PDF resource storage.

3.5 Chosen Architecture Pattern and Rationale

Layered modular monolith chosen for clear separation of presentation, API, domain modules, and persistence while keeping the project suitable for a single deployment.

3.6 Technology Stack & Data Stores

React.js, Node.js, Express.js, MongoDB, REST APIs, and configured PDF file storage. HTTPS is used for sensitive communication in the deployed environment.

3.7 Risks & Mitigations

Risk: invalid/oversized uploads, data loss, API mismatch, security defects, and concurrent-user issues → Mitigation: server-side validation, MongoDB persistence checks, agreed API contracts, RBAC/security controls, and project-level testing.

3.8 Traceability to Requirements

CSMQ-F-001/F-002/F-004 → Auth UI/API + User model + authentication

CSMQ-F-010/F-012/F-013/F-041 → Study Material + File Storage + Admin

CSMQ-F-020/F-022/F-042 → Question Bank + Admin

CSMQ-F-030/F-032 → Quiz module

CSMQ-F-033/F-034/F-035/F-036/F-040 → Attempt, Evaluation, History, Performance modules.

3.9 Security Architecture

Threat Modeling (STRIDE): Spoofing → authentication; Tampering → RBAC and validation; Information Disclosure → response filtering and HTTPS; DoS → upload/request controls; Elevation of Privilege → backend role checks.

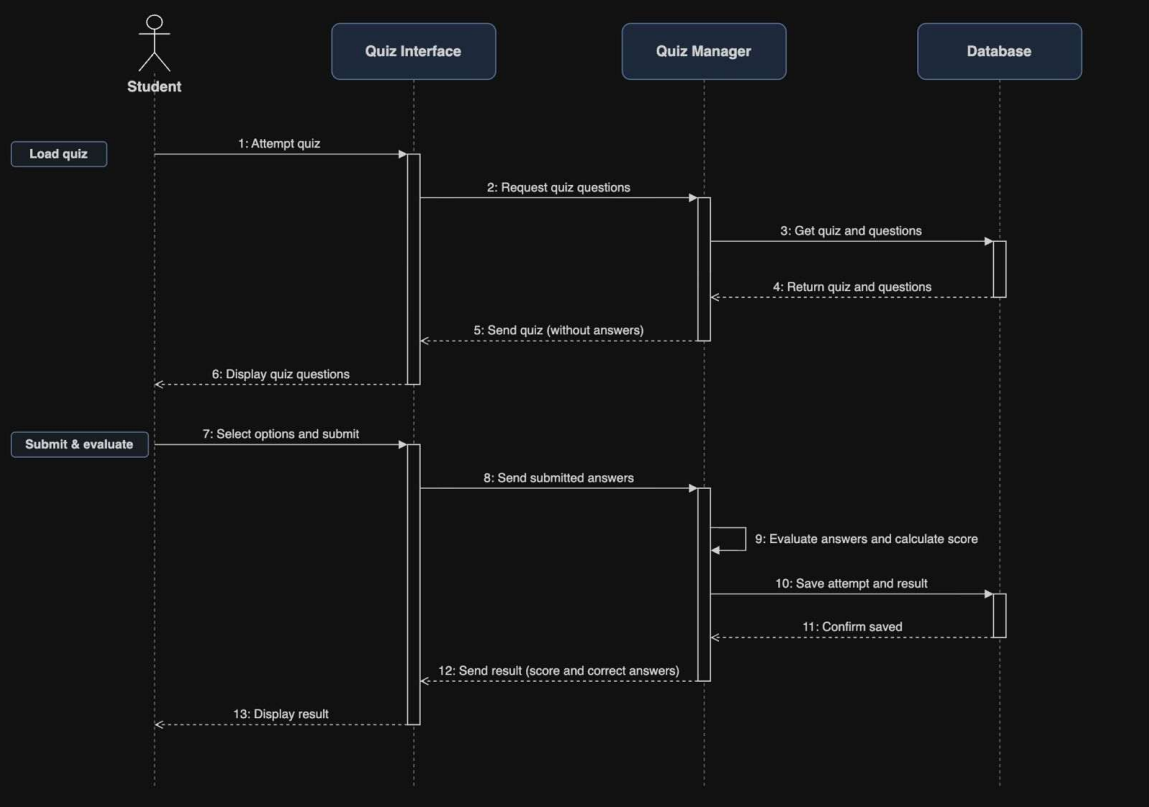
4. Design

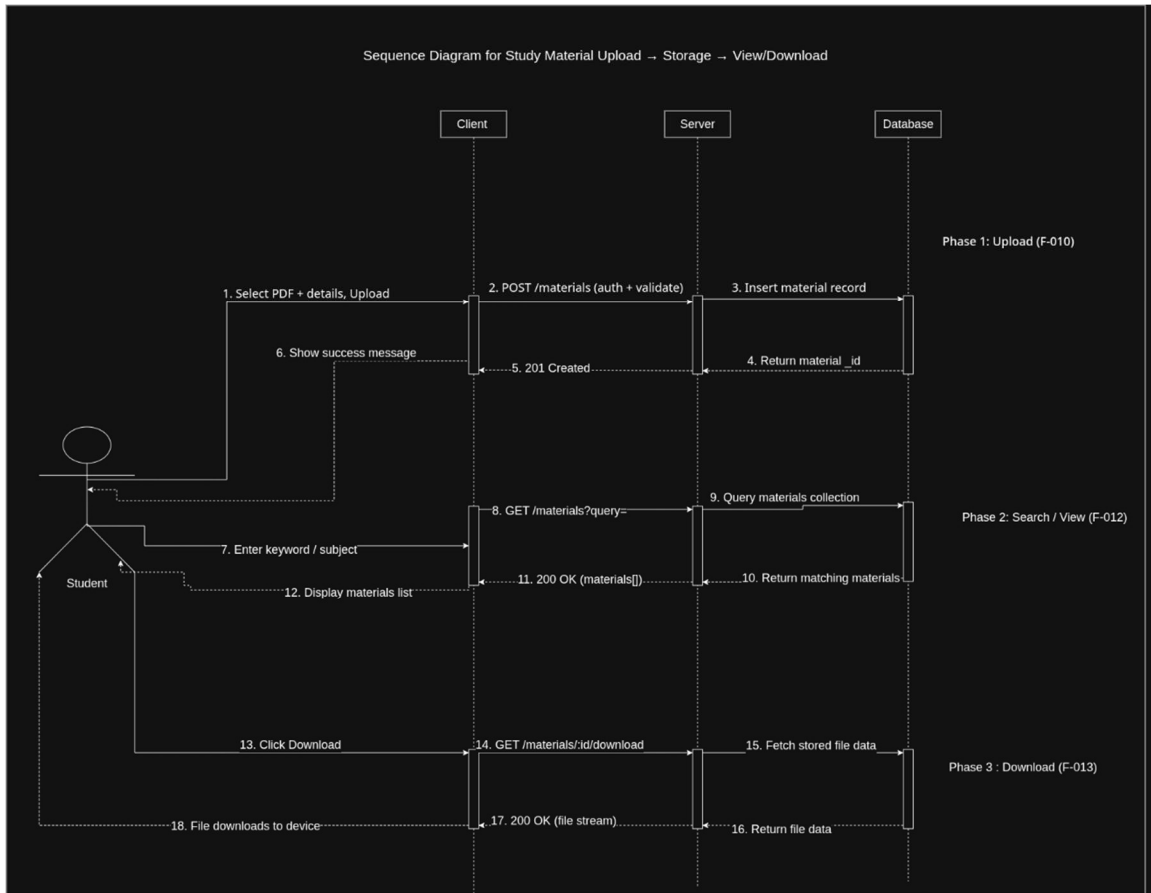
4.1 Design Overview

The design follows the SRS functional modules. Each module owns its related UI/API/data logic, while shared authentication, authorization, integration, and deployment concerns remain common.

4.2 UML Sequence Diagrams

The sequence diagrams illustrate two flows: (1) quiz attempt, automatic evaluation, and result display; and (2) study-material upload, search/view, and download. Authentication is a precondition for protected operations.





4.3 API Design

Interface definitions for Auth and Study Material components, with a quiz-attempt interface also shown.

Endpoint: `/api/auth/login`

Method: POST

Request: { email, password }

Response: { authenticatedUser, token/session }

Errors: 400 validation, 401 invalid credentials.

Endpoint: `/api/materials`

Method: POST

Request: multipart PDF + title, subject, description

Response: created material metadata + file reference

Errors: 400 validation, 413 oversized upload.

Endpoint: `/api/attempts`

Method: POST

Request: { quizId, responses[] }

Response: { score, percentage, resultId, correctAnswers }

Correct answers for the submitted quiz are returned only after successful submission and evaluation.

4.4 Error Handling, Logging & Monitoring

Use standardized error responses with safe messages. Do not include passwords, authentication tokens, password hashes, or correct answers in error messages or logs. Correct answers must not be disclosed to students before quiz submission. After successful submission and evaluation, correct answers for the submitted quiz may be included in the result, as shown in the quiz sequence diagram. Log safe request/event context and monitor failures, response times, uploads, quiz submissions, and application health.

4.5 UX Design

Responsive web UI with clear navigation, validation, success/error feedback, loading and empty states, and role-appropriate screens for students and administrators.

4.6 Open Issues & Next Steps

Next steps: complete integration, cross-test all modules, verify NFRs/security acceptance criteria, and prepare the final deployment/demo package.

5. Appendices

5.1 Glossary: SAD, SRS, UI, API, REST, RBAC, MCQ, MERN, MongoDB.

5.2 References: Project SRS, Team Work Division document.

5.3 Tools: PlantUML, draw.io.